

Transformation et manipulation de donnees



Dr Khadidja Henni

Séance 12: Extraction des caractéristiques des séries temporelles.

Plan de cours

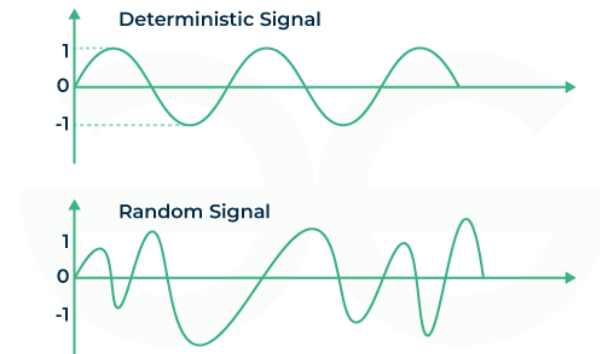
- Introduction à la transformation de signaux
 - Caractéristiques temporelles classiques
 - Caractéristiques fréquentielles
 - Caractéristiques temps-fréquence
 - Techniques avancées
-
- Laboratoire formatif

Analyse et transformation des signaux

Introduction

Qu'est-ce qu'un signal ?

- Un signal est une fonction mathématique représentant l'évolution d'une grandeur physique dans le temps ou l'espace.
- Signaux temporels : variation d'une grandeur en fonction du temps (ECG, EEG, audio, vibrations, etc.).
- Signaux spatiaux : variation dans l'espace (image satellite, IRM, etc.).
- Types de signaux :
 - Discrets (échantillonnés) : ex. : signaux numériques stockés.
 - Continus : ex. : signal mesuré en temps réel avant numérisation.
 - Stationnaires : propriétés statistiques constantes dans le temps (rare).
 - Non-stationnaires : propriétés évolutives (ex. : EEG avec sommeil)



Signals in Electrical Circuits

<https://www.geeksforgeeks.org/electrical-engineering/signals/>

Introduction

Pourquoi transformer un signal ?

Les signaux bruts sont souvent :

- **bruyants** (présence d'artefacts ou parasites),
- **complexes** à interpréter dans leur forme initiale,
- **hauts en dimension** (difficiles à exploiter directement).

Objectif de la transformation : extraire des **informations discriminantes** pour faciliter les tâches de classification, détection ou régression.

Exemple : Un électrocardiogramme (ECG) de 10 secondes à 500 Hz donne 5 000 points — au lieu d'utiliser directement ces 5 000 valeurs, on préfère extraire : rythme cardiaque, énergie, pic R, etc.

Introduction

Principaux domaines d'application

Domaine	Signal typique	Objectif de la transformation
Biomédical (ECG/EEG/EDA,...)	Signaux physiologiques	Diagnostic ou détection de pathologies
Audio	Voix, musique	Reconnaissance vocale, détection d'émotions
Industriel	Vibrations machines	Maintenance prédictive
Transport	Accéléromètres	Détection de chocs ou comportements

Workflow complet de traitement des signaux avant l'extraction de caractéristiques

1. Acquisition du signal brut

- Récupération des données depuis :
 - Un capteur (EEG, accéléromètre...),
 - Un fichier (ex. : .edf, .wav, .csv),
 - Une API ou une base de données (MIMIC, PhysioNet, etc.)
- Vérification du **taux d'échantillonnage** (ex. : 256 Hz, 512 Hz)
- Stockage : vecteurs temporels (1D) ou multicanal (2D)

Exemples de bibliothèques :

- EEG/ECG : mne, wfdb, biosppy
- Audio : librosa, soundfile, torchaudio
- Générique : pandas, numpy

Workflow complet de traitement des signaux avant l'extraction de caractéristiques

2. Prétraitement du signal

a) Filtrage

- **Passe-bas** : supprimer les hautes fréquences (ex. : >40 Hz EEG)
- **Passe-haut** : supprimer les dérives lentes (ex. : <0.5 Hz ECG)
- **Notch filter** : supprimer bruit secteur 50/60 Hz

b) Suppression d'artefacts / bruit

- ICA (independent component analysis) pour EEG
- Denoising spectral pour audio

c) Normalisation / centrage

- Centrer le signal autour de 0
- Échelle [0,1] ou z-score (souvent pour réseaux de neurones)

d) Interpolation ou resampling

- Recalage temporel ou harmonisation du taux d'échantillonnage

e) Détection de valeurs aberrantes

- Clipping, imputation, lissage

```
from scipy.signal import butter, filtfilt

def lowpass_filter(data, cutoff=40, fs=256, order=5):
    nyq = 0.5 * fs # fréquence de Nyquist
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    y = filtfilt(b, a, data)
    return y
```

```
filtered = lowpass_filter(signal, cutoff=40, fs=fs)
```

```
import mne

raw = mne.io.read_raw_edf("fichier.edf", preload=True)
raw.filter(l_freq=0.5, h_freq=40) # Passe-bande entre 0.5 et 40 Hz
raw.notch_filter(freqs=50) # Filtre notch 50 Hz
```

Workflow complet de traitement des signaux avant l'extraction de caractéristiques

3. Segmentation du signal

a) Segmentation fixe

- Fenêtres de taille fixe (ex. : 4 s, 1024 points)
- Recouvrement (ex. : 50 %)

b) Segmentation basée sur événements

- Crises, pics, silences, cycles

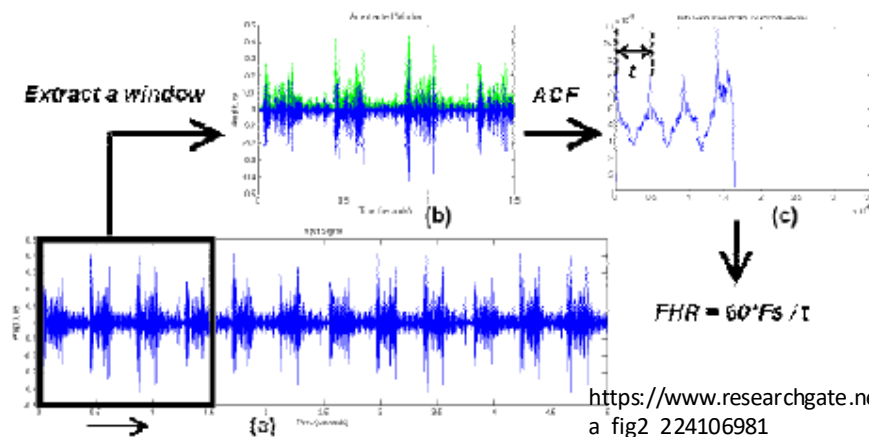
Exemples de bibliothèques :

numpy, scipy, librosa.util.frame, mne.Epochs, pyts, tslearn

```
import numpy as np

def segment_signal(signal, window_size, step):
    """
    Segmente un signal 1D en fenêtres fixes avec recouvrement.
    :param signal: le signal d'entrée (1D)
    :param window_size: taille de la fenêtre (en points)
    :param step: décalage entre les fenêtres (overlap = window_size - step)
    :return: tableau 2D (nb_fenêtres, taille_fenêtre)
    """
    segments = []
    for start in range(0, len(signal) - window_size + 1, step):
        segment = signal[start:start + window_size]
        segments.append(segment)
    return np.array(segments)

# Exemple d'utilisation
fs = 256 # Hz
window_size = 4 * fs # 4 secondes
step = 2 * fs # 50 % de recouvrement
segments = segment_signal(signal, window_size, step)
print(segments.shape) # (nb_segments, 1024)
```



Workflow complet de traitement des signaux avant l'extraction de caractéristiques

3. Segmentation du signal

b) Segmentation basée sur événements

- Crises, pics, silences, cycles
- C'est une méthode où l'on découpe le signal **selon des événements spécifiques détectés automatiquement ou annotés manuellement**, plutôt qu'en fenêtres fixes.

Isoler des portions pertinentes du signal autour d'un événement d'intérêt.

- Quand l'utiliser : quand les événements sont clairs ou annotés (crises, battements, transitions).
- Avantage : centrage précis autour d'événements cliniques ou physiques.
- Inconvénient : dépend fortement de la qualité de la détection.

Workflow complet de traitement des signaux avant l'extraction de caractéristiques

4. Contrôle qualité / visualisation (optionnelle mais recommandée)

- Affichage de segments choisis
- Vérification de la qualité du signal segmenté
- Statistiques descriptives (valeurs moyennes, histogrammes)

Exemples :

- `matplotlib`, `seaborn`, `plotly`, `mne.plot`, `librosa.display.specshow`

Workflow complet de traitement des signaux avant l'extraction de caractéristiques

- 5. Extraction de caractéristiques

C'est à **partir de ce point** que commencent les méthodes de **feature extraction**, on a **3 formes principales de transformation + Apprentissage de représentations** :

- Temporelles (moyenne, RMS, ZCR...)
 - Fréquentielles (FFT, PSD, centroïde...)
 - Temps-fréquence (STFT, CWT, MFCC...)
-
- Apprentissage de représentations (PCA, autoencodeur, etc.)

Caractéristiques temporelles classiques

Caractéristiques temporelles classiques

Les **caractéristiques temporelles** sont extraites directement **dans le domaine du temps**, c'est-à-dire à partir de la forme brute du signal (sans transformation fréquentielle).

Pourquoi utiliser des caractéristiques temporelles ?

- Simples à calculer et interpréter
- Rapides à extraire
- Représentent **l'amplitude, la forme, la variabilité et l'énergie** du signal dans un segment donné
- Très utiles pour des signaux **stationnaires ou faiblement bruités**.

Quand les utiliser ?

- Pour une **première étape d'analyse exploratoire**
- Lorsque les signaux ont peu de variabilité fréquentielle
- Pour des **modèles simples ou interprétables**
- En combinaison avec des caractéristiques fréquentielles (multimodal)

Caractéristiques temporelles classiques

Les principales **caractéristiques temporelles** sont :

Feature	Description	Utilité
Moyenne (mean)	Valeur moyenne des amplitudes dans une fenêtre	Centrage global du signal
Écart-type (std)	Dispersion autour de la moyenne	Variabilité
Maximum / Minimum	Valeurs extrêmes	Amplitude maximale
RMS (Root Mean Square)	Énergie moyenne du signal (puissance)	Évaluation de l'intensité globale
Zero Crossing Rate (ZCR)	Nombre de fois où le signal change de signe (passe par zéro)	Indicateur de fréquence ou "vibration"
Skewness	Asymétrie de la distribution	Détection de biais dans l'onde
Kurtosis	Aplatissement (ou pics) dans la distribution	Sensible aux valeurs extrêmes
Signal Entropy	Mesure du désordre / imprévisibilité	Utilisé pour signaux physiologiques

Caractéristiques temporelles classiques

Exemple d'utilisation:

Domaine	Application	Feature utilisée
ECG	Détection de battements anormaux	RMS, pics, skewness
Audio	Voix vs bruit	ZCR, énergie
EEG	Classification veille/sommeil	Écart-type, entropie
Vibrations	Maintenance préventive	RMS, kurtosis, ZCR

Caractéristiques temporelles classiques

```
import pandas as pd
from scipy.stats import skew, kurtosis
import numpy as np

def extract_temporal_features(segment):
    zcr = ((segment[:-1] * segment[1:]) < 0).sum()
    return {
        'mean': np.mean(segment),
        'std': np.std(segment),
        'max': np.max(segment),
        'min': np.min(segment),
        'rms': np.sqrt(np.mean(segment ** 2)),
        'zcr': zcr,
        'skewness': skew(segment),
        'kurtosis': kurtosis(segment)
    }

# Signal simulé
fs = 256 # Hz
signal = np.sin(2 * np.pi * 1 * np.arange(0, 10, 1/fs)) + 0.2*np.random.randn(2560)

# Fenêtrage
window_size = 512
step = 256 # 50% recouvrement

features_list = []
for i in range(0, len(signal) - window_size + 1, step):
    segment = signal[i:i + window_size]
    features = extract_temporal_features(segment)
    features_list.append(features)

# ✓ Conversion en DataFrame
features_df = pd.DataFrame(features_list)

# Affichage
features_df.head()
```

Nombre de segments extraits : 9

	mean	std	max	min	rms	zcr	skewness	kurtosis
0	-0.007993	0.736702	1.417234	-1.349084	0.736746	44	-0.031355	-1.308818
1	-0.000566	0.736973	1.447896	-1.408537	0.736973	39	0.002467	-1.278437
2	0.005837	0.735827	1.447896	-1.408537	0.735850	35	0.013725	-1.322980
3	0.010710	0.737693	1.562507	-1.460720	0.737771	38	-0.005586	-1.290465
4	0.005093	0.728970	1.655201	-1.460720	0.728988	42	0.007559	-1.221463

Caractéristiques temporelles classiques

Avantages

- Simples à interpréter et implémenter
- Très peu coûteuses en calcul
- Peu sensibles à la qualité de segmentation
- Utiles en contexte temps réel (embarqués)

Inconvénients

- Ne capturent **pas la structure fréquentielle** (inutile si les infos sont en fréquence)
- Peu discriminantes si le signal a peu de variance d'amplitude
- Peu robustes au **bruit** ou **artefacts**
- Moins efficaces sur des signaux **non stationnaires**

Caractéristiques fréquentielles

Caractéristiques fréquentielles

- La **fréquence** mesure **combien de fois un motif se répète en une seconde** dans un signal. Elle est exprimée en **Hertz (Hz)**.
- Les **caractéristiques fréquentielles** sont extraites après avoir transformé le signal du **domaine temporel vers le domaine fréquentiel** (généralement via la FFT).
- Application à la FFT (Transformée de Fourier)
 - Quand on applique la FFT, elle décompose le signal en plusieurs fréquences pures :
Chaque fréquence trouvée est associée à une amplitude → cela forme le spectre du signal.

Pourquoi utiliser les caractéristiques fréquentielles ?

- Les signaux complexes (EEG, audio, vibrations, etc.) **portent souvent leur information dans la répartition spectrale**.
- Elles permettent d'analyser :
 - La **composition harmonique** (audio, parole)
 - Les **bandes de fréquences dominantes** (EEG)
 - Les **pannes mécaniques** (fréquences anormales dans les vibrations)

Caractéristiques fréquentielles

Quand les utiliser ?

À utiliser quand...	Exemples
Le signal est non stationnaire	EEG avec phases de sommeil, audio avec parole
Il existe des bandes fréquentielles utiles	EEG (alpha, beta...), bruit secteur
On veut capturer la forme énergétique	Audio, vibrations
On a besoin de classifier le contenu spectral	Voix vs bruit, machine saine vs endommagée

Caractéristiques fréquentielles

Principales caractéristiques fréquentielles

Feature	Description	Utilité principale
Spectral Centroid	"Centre de gravité" du spectre	Tonalité moyenne du signal
Spectral Bandwidth	Écart-type du spectre autour du centroïde	Richesse harmonique
Spectral Rolloff	Fréquence sous laquelle X% (ex. 85%) de l'énergie est contenue	Distribution énergétique
Spectral Flatness	Indique si le spectre est plat (bruit blanc) ou structuré (harmoniques)	Indice de tonalité ou de bruit
Spectral Entropy	Désordre du spectre	Variabilité énergétique
Fréquences dominantes	Pics de fréquence (via FFT)	Détection de défauts mécaniques
Bande de puissance	Énergie dans des bandes fixes (delta, alpha...)	EEG, analyse biomédicale

Caractéristiques fréquentielles

```
import librosa
import numpy as np
import pandas as pd

# Charger le fichier audio
y, sr = librosa.load("audio.wav", sr=16000)

# Paramètres de segmentation
frame_length = 2048      # ≈ 128 ms à 16 kHz
hop_length = 1024        # ≈ 50 % overlap

# Extraire les features segment par segment
centroid = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=frame_length, hop_length=hop_length)[0]
bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr, n_fft=frame_length, hop_length=hop_length)[0]
rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr, n_fft=frame_length, hop_length=hop_length, roll_percent=0.85)[0]
flatness = librosa.feature.spectral_flatness(y=y, n_fft=frame_length, hop_length=hop_length)[0]

# Construire un DataFrame
features_df = pd.DataFrame({
    'spectral_centroid': centroid,
    'spectral_bandwidth': bandwidth,
    'spectral_rolloff': rolloff,
    'spectral_flatness': flatness
})

# Affichage des premières lignes
features_df.head()
```

	spectral_centroid	spectral_bandwidth	spectral_rolloff	spectral_flatness
0	2159.276639	2387.832137	5492.1875	0.015059
1	2898.245235	2477.926814	6156.2500	0.011997
2	2992.785453	2518.881513	6226.5625	0.012254
3	2949.631024	2508.481860	6257.8125	0.011185
4	3019.948220	2548.401931	6328.1250	0.010682

Caractéristiques fréquentielles

Avantages

- Représentation **puissante** pour les signaux riches en contenu spectral (audio, EEG)
- Permet de **détecter des anomalies** ou des signatures spectrales précises
- Bien adaptées aux modèles de classification

Inconvénients

- Perte d'information temporelle → ne permet pas de localiser *quand* une fréquence apparaît
- Sensibles au **bruit** si pas bien prétraité
- Requiert souvent **une bonne fenêtre de segmentation**

Caractéristiques temps-fréquence

Caractéristiques temps-fréquentielles

- Les **caractéristiques temps-fréquentielles** sont obtenues en analysant **l'évolution des fréquences au cours du temps**, via des techniques comme la **STFT** ou la **transformée en ondelettes**.

Pourquoi utiliser les caractéristiques temps-fréquentielles ?

- Parce que de nombreux signaux sont **non stationnaires**, c'est-à-dire que leurs composantes fréquentielles changent dans le temps.
- L'analyse uniquement dans le domaine fréquentiel (FFT) **perd la localisation temporelle**.
- Ces caractéristiques permettent de capturer **“quoi se passe, quand”**.

Caractéristiques temps-fréquentielles

Quand les utiliser?

Situation	Exemples
Fréquences changent rapidement	Parole, EEG de crise, alarme
Comportement transitoire ou épisodique	Choc, impulsion, activité neuronale
Signaux avec structure cyclique variable	Respiration, voix émotionnelle
Détection d'événements dans un spectre	Bruit soudain, défaillance mécanique

Techniques de représentation

Méthode	Description
STFT	Découpe le signal en fenêtres puis applique la FFT
Spectrogramme	Représentation graphique issue de la STFT (temps vs fréquence vs énergie)
CWT (ondelettes)	Analyse multi-résolution avec des ondelettes (utile pour EEG/EMG)
MFCC (audio)	Représentation compacte basée sur les spectres de type mél

Caractéristiques temps-fréquentielles

Exemples de caractéristiques temps-fréquentielles

Caractéristique	Description	Utilité principale
Spectrogramme	Représentation du signal dans le temps-fréquence	Visualisation globale du contenu
MFCC (audio)	Coefficients cepstraux de type mél	Reconnaissance vocale, émotion
Wavelet coefficients	Transformée en ondelettes	Crises EEG, pics rapides, compressions
Spectral contrast	Écart entre pics et creux dans le spectre	Richesse harmonique
Chroma energy	Énergie par “note” musicale (audio)	Classification musicale

Caractéristiques temps-fréquentielles

Avec Librosa (audio)

```
import librosa
import numpy as np
import pandas as pd

# Charger un signal audio
y, sr = librosa.load("audio.wav", sr=16000)

# Extraire des MFCCs segment par segment
mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, hop_length=512)

# Transposer pour mettre chaque vecteur MFCC en ligne
mfccs_df = pd.DataFrame(mfccs.T, columns=[f'mfcc{i+1}' for i in range(mfccs.shape[0])])

mfccs_df.head()
```

	mfcc1	mfcc2	mfcc3	mfcc4	mfcc5	mfcc6	mfcc7	mfcc8	mfcc9	mfcc10	mfcc11	mfcc12	mfcc13
0	-28.707703	56.785107	38.787842	13.613803	-3.109254	-14.818867	-25.722424	-23.188023	-18.489048	-7.320229	-5.018888	1.623280	13.866402
1	-13.719508	29.774792	20.726147	5.536539	-4.904796	-10.785646	-22.440060	-20.881683	-17.872416	-11.650960	-7.592983	1.549814	11.667480
2	-24.579956	9.528799	7.056514	4.202565	-5.308248	-8.278343	-16.196314	-15.374060	-12.145798	-9.285192	-9.171201	-2.223363	8.140579
3	-24.886574	9.109200	8.727789	3.165389	-4.936071	-7.487846	-14.730072	-16.534033	-15.195065	-9.540992	-2.784923	1.293502	8.810736
4	-26.389343	6.438810	9.210227	5.422942	-1.541729	-4.669238	-12.424542	-12.128841	-10.409317	-5.939115	-0.752476	3.964123	9.253576

Caractéristiques temps-fréquentielles

Exemple avec `scipy.signal.spectrogram` pour EEG ou vibration

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal import spectrogram
from scipy.stats import entropy

# 1. Générer un signal simulé (2 fréquences dominantes)
fs = 256 # fréquence d'échantillonnage (Hz)
t = np.linspace(0, 10, fs * 10, endpoint=False) # 10 secondes
signal = np.sin(2 * np.pi * 10 * t) + 0.5 * np.sin(2 * np.pi * 25 * t)

# 2. Calcul du spectrogramme
f, t_spec, Sxx = spectrogram(signal, fs=fs, nperseg=256, noverlap=128)

# 3. Extraction des caractéristiques par fenêtre temporelle
features_list = []

for i in range(Sxx.shape[1]): # chaque colonne = un segment temporel
    segment_power = Sxx[:, i]
    segment_power_norm = segment_power / np.sum(segment_power + 1e-12) # pour entropie

    # Fréquence du pic de puissance
    freq_max = f[np.argmax(segment_power)]

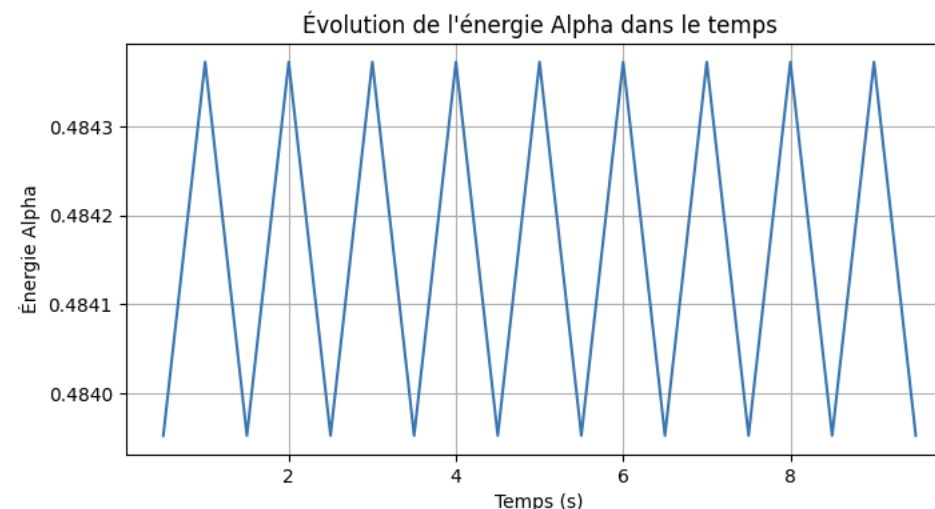
    # Énergie dans les bandes EEG classiques
    def band_energy(f_min, f_max):
        mask = (f >= f_min) & (f < f_max)
        return segment_power[mask].sum()

    features = {
        'power_mean': np.mean(segment_power),
        'power_std': np.std(segment_power),
        'freq_max_power': freq_max,
        'entropy': entropy(segment_power_norm),
        'band_energy_delta': band_energy(1, 4),
        'band_energy_theta': band_energy(4, 8),
        'band_energy_alpha': band_energy(8, 13),
        'band_energy_beta': band_energy(13, 30)
    }
    features_list.append(features)

# 4. Conversion en DataFrame
features_df = pd.DataFrame(features_list)

# 5. Affichage

# 6. (Optionnel) Visualisation de l'énergie alpha dans le temps
plt.figure(figsize=(8, 4))
plt.plot(t_spec, features_df['band_energy_alpha'])
plt.title("Évolution de l'énergie Alpha dans le temps")
plt.xlabel("Temps (s)")
plt.ylabel("Énergie Alpha")
plt.grid(True)
plt.show()
features_df.head()
```



	power_mean	power_std	freq_max_power	entropy	band_energy_delta	band_energy_theta	band_energy_alpha	band_energy_beta
0	0.004838	0.040885	10.0	1.005992	0.000030	0.007838	0.483953	0.132018
1	0.004851	0.040952	10.0	1.010916	0.000029	0.007912	0.484373	0.133203
2	0.004838	0.040885	10.0	1.005992	0.000030	0.007838	0.483953	0.132018
3	0.004851	0.040952	10.0	1.010916	0.000029	0.007912	0.484373	0.133203
4	0.004838	0.040885	10.0	1.005992	0.000030	0.007838	0.483953	0.132018

Caractéristiques temps-fréquentielles

Type de caractéristique	Méthode utilisée	Domaine typique
MFCCs	librosa.feature.mfcc	Audio / NLP
Spectrogramme	scipy.signal.spectrogram, librosa.stft	EEG, audio, vibration
Contraste spectral	librosa.feature.spectral_contrast	Voix, musique
Ondelette continue (CWT)	pywt.cwt	EEG, EMG, vibrations

Caractéristiques temps-fréquentielles

Avantages

- Permet une **analyse complète** du contenu temps-fréquence
- Adaptée aux **signaux non stationnaires**
- Permet la détection **fine et localisée d'événements**
-

Inconvénients

- **Plus coûteuse en calcul**
- Choix des paramètres (taille fenêtre STFT, ondelette, etc.) sensibles
- Résolution limitée par compromis temps/fréquence (STFT)

Ce qui ce fais actuellement

Extraction automatique des caractéristiques

Objectif: apprendre automatiquement une représentation pertinente du signal brut, sans avoir à coder manuellement les caractéristiques (moyenne, RMS, FFT...).

Pourquoi passer au deep learning pour l'extraction de caractéristiques ?

- Les signaux réels (EEG, ECG, audio...) sont **complexes, bruités, non stationnaires**, avec des **structures cachées non linéaires**.
- Les méthodes traditionnelles (RMS, PCA, FFT) sont **limitées** : elles capturent uniquement ce qu'on leur demande.
- Le deep learning apprend **de manière automatique** des représentations :
 - compactes
 - optimisées pour la tâche (ex. : classification)
 - non linéaires
 - adaptables à différents types de signaux (1D, 2D, multicanaux...)

Méthodes profondes utilisées pour extraire des caractéristiques (features)

1. CNN sur signal brut (1D)

- Appliqué directement sur des signaux EEG/ECG segmentés
- Les couches de convolution apprennent des **filtres discriminants**
- Pas besoin de transformation en spectrogramme
- Très utilisé en :
 - **diagnostic médical (ECG, EEG)**
 - **détection d'événements audio**

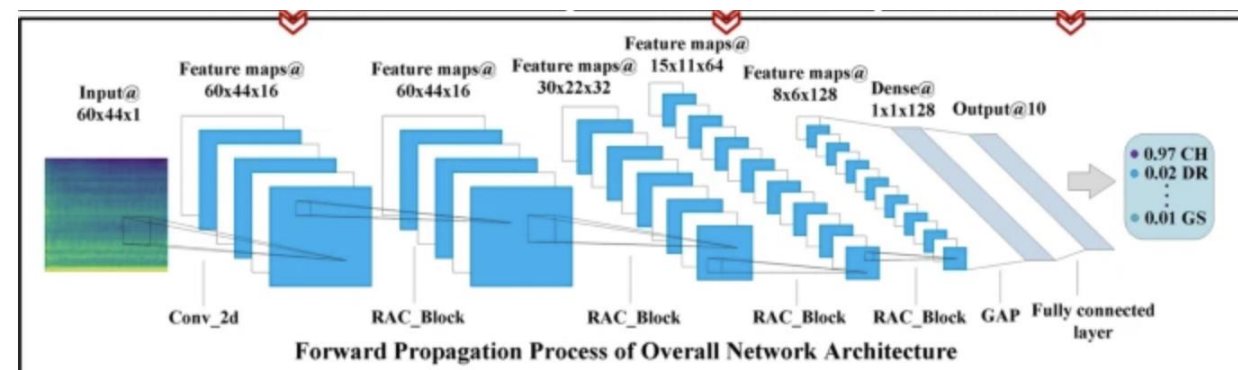
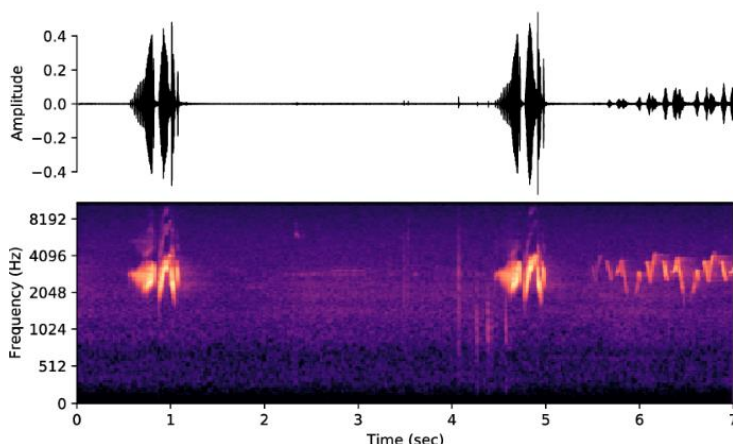
2. CNN sur spectrogrammes (2D)

- On transforme le signal avec une **STFT** ou **CWT**
- Le spectrogramme (temps × fréquence) devient une **image**
- Un **CNN 2D** (ex. : ResNet, VGG, EfficientNet) extrait automatiquement des patterns visuels

Très populaire en :

- **reconnaissance vocale, émotions, détection de crises**
- **analyse bioacoustique** (battements cardiaques, respiration)

Exemple : Deep ConvNets for EEG decoding



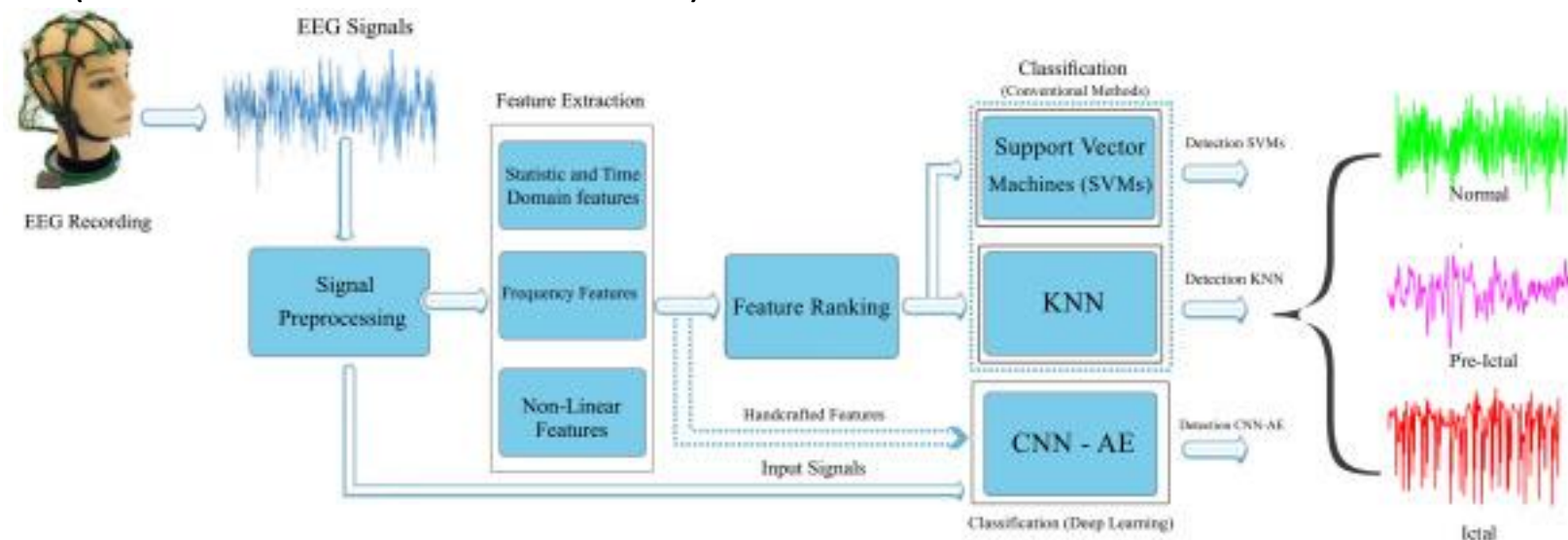
Méthodes profondes utilisées pour extraire des caractéristiques (features)

3. Autoencodeurs profonds (non supervisés)

- Le signal est encodé dans une **représentation latente compacte**
- L'autoencodeur apprend à **reconstruire** l'entrée
- La **couche centrale** représente un bon "résumé" du signal

Utilisé pour :

- **compression, dénormalisation, prétraitement pour SVM**
- **détection d'anomalies** (anomalous reconstruction error)



Extraction automatique des caractéristiques

4. LSTM / GRU / RNN

- Appliqués sur signaux séquentiels (EEG, respiration, ECG)
- Capables de **mémoriser des dépendances temporelles**
- Les **vecteurs cachés** ou **états finaux** sont utilisés comme caractéristiques

Utilisé en :

- **prédiction de séquences**
- **analyse de cycles (cardiaques, respiration, sommeil)**

5. Transformers (avec ou sans supervision)

- Modèles comme **TimeTransformer**, **EEG-BERT**, **MiniRocket + Transformer**
- Apprennent une **représentation globale du signal** via des mécanismes d'attention
- Excellents pour :
 - Signaux longs
 - Données multicanaux (EEG à 64 électrodes)
 - Apprentissage auto-supervisé

Exemples récents :

- EEG-BERT
- Time2Vec + Transformer for multivariate signals

Laboratoire formatif

Laboratoire formatif

Étapes :

1. Charger les fichiers .wav avec librosa
2. Découper chaque audio en segments de 2s avec 50 % d'overlap (utilisez des boucles avec range() ou librosa.util.frame())
3. Étiqueter chaque segment à partir du CSV, labels.csv
4. Extraire des features :
 - Temporelles : RMS, ZCR, énergie. (librosa.feature.rms(), librosa.feature.zero_crossing_rate(), np.mean(segment**2))
 - Fréquentielles : centroïde spectral, bande passante, rolloff, MFCC (librosa.feature.spectral_centroid(), librosa.feature.spectral_bandwidth(), librosa.feature.spectral_rolloff(), librosa.feature.mfcc() (juste les deux premiers coefficients mfcc[0] et mfcc[1]))
6. Former un classifieur (SVM, RF, ou Naive Bayes)
7. Évaluer le modèle (accuracy, recall, F1)
8. Comparer différents jeux de features